# AgentForge

## Clinical Co-Pilot

Building Production-Ready AI Agents for Healthcare

**Project Requirements Document**

Gauntlet AI — Austin Admission Track

# How to Use This Case Study

This case study is your north star for the duration of this project. You are required to use it as the foundation for every decision you make: what you build, what you prioritize, and what you expand. It does not define the ceiling. If you see opportunities to go deeper or broader, take them, but do it in the spirit of the case study, not in spite of it.

What this case study does define is the floor: every feature, every architectural decision, and every tradeoff you make should be traceable back to the problem of a customer needing reliable, fast, secure access to data. Use this document as a reference, a constraint, and a lens.

The decisions you make this week are the foundation you will build on in weeks two and three — good architecture will compound; technical debt will cost double later. You will be evaluated on your thoroughness, your thoughtfulness, your creativity, and your ability to leverage technology to build something viable.

# The Scenario

A physician has 90 seconds between patient rooms. In that window, they need to recall who they're seeing, why, what changed since the last visit, what's on file, and what actually matters today. Right now, that means scanning dense EHR notes, flipping through lab results, cross-referencing medication lists, and doing it all under pressure while the patient is already waiting.

Your task is to build a Clinical Co-Pilot — an AI agent embedded directly into OpenEMR that gives a physician the context they need, the moment they need it. Not a chatbot that answers generic medical questions. A system that knows this patient, their history, their meds, their recent labs, and can surface what's relevant to today's visit in a conversation-style interface.

| | |
|---|---|
| **WHY THIS MATTERS** | A confidently stated hallucination in a clinical setting doesn't just damage trust — it can directly harm a patient. The gap between a prototype that works in a demo and an agent that can be trusted in a hospital is the entire scope of this project. |

# The Hard Problems

This is not a build-whatever-you-want project. The case study surface area is intentionally constrained, but the engineering problems within it are real and unsolved. Your job is to grapple with them honestly.

## Authorization & Access Control

Who is allowed to query patient data? A physician has access to their own patients. A nurse may have different permissions. A resident may be supervised. The system must know who is

asking and enforce appropriate access — not assume all users are trusted. Multi-user environments are the norm in clinical settings, and your architecture must reflect that.

### Verification & Trust

Every claim the agent makes must be traceable back to a source in the patient's actual record. If the agent says a patient is on a specific medication, that statement must be grounded in the data — not inferred, not assumed, not hallucinated. Equally, the agent must be aware of domain constraints: clinical rules, dosage thresholds, interaction flags. A response that violates what the underlying data actually says is a failure, not a feature.

How you implement this is up to you. There are many approaches. The requirement is that you have one, that it is deliberate, and that you can defend it.

### Speed vs. Completeness

A physician walking into a room needs an answer in seconds, not minutes. But a complete answer might require pulling from multiple data sources, running multiple tool calls, and synthesizing conflicting records. How you manage that tradeoff — what you prioritize, what you defer, how you communicate uncertainty — is a core design decision you will need to make explicitly.

### Data Security & HIPAA

Patient health information is Protected Health Information (PHI) under HIPAA. This is not optional compliance — it is a legal and ethical constraint that shapes every architectural decision: how data is stored, transmitted, logged, and who can access it. Your audit and your architecture documentation must demonstrate that you understand these constraints, not just that you know the acronym.

**Note:** Only use demo data with this codebase. For all Gauntlet projects, act as if you have a signed Business Associate Agreement with all LLM providers that no data will be used for training purposes.

### Failure Modes

What happens when a tool fails? When a patient record is incomplete? When the model returns something unexpected? A clinical tool that crashes or silently fails is worse than no tool at all. Graceful degradation, transparent errors, and predictable behavior under failure conditions are not nice-to-haves.

---

## The Codebase: OpenEMR

You will fork OpenEMR — a widely-deployed, open-source Electronic Health Record system with a large, real codebase. This is your foundation. You are not building a clinical app from scratch; you are integrating an AI agent into existing healthcare infrastructure.

The codebase will likely be unfamiliar. Part of this project is demonstrating that you can orient yourself in a large, complex system, understand its architecture, identify where your work fits, and integrate cleanly rather than bolt something on.

**Fork from:** https://github.com/openemr/openemr

| | |
|---|---|
| **GATE** | Project completion and interviews are both required for Austin admission. The audit is a hard gate — you must complete it before building the AI layer. |

# Project Schedule

**Hard Gates.** This is a one-week sprint with four checkpoints. All times are Central (Austin)

| Checkpoint | Deadline | Focus |
|---|---|---|
| **Architecture Defense** | 24 hours | Architecture research and planning |
| **MVP** | Tuesday @ 11:59PM | App audit, agent plan, deployed app, and demo video. AI Interview required 24 hours after submission. |
| **Early Submission** | Thursday @ 11:59PM | A deployed agent, eval framework in place, observability wired in, and demo video. AI Interview required 24 hours after submission. |
| **Final** | Sunday @ Noon | Production-ready agent, demo video, and social media post. AI Interview required 24 hours after submission. |

# MVP: Recommended Steps

The MVP is not a working agent. It is the foundation that makes a trustworthy agent possible.

| Stage | Name | Deliverable |
|---|---|---|
| **1** | **Run It Locally** | OpenEMR running in your local environment with sample patient data |
| **2** | **Deploy It** | Publicly accessible deployment of your OpenEMR fork |
| **3** | **Audit It** | Full audit with a written record of any findings |

| Stage | Name | Deliverable |
|---|---|---|
| 4 | **Identify Users** | A breakdown of the users your app addresses and their use cases |
| 5 | **Plan the Agent** | A concrete, codebase-informed plan for how you will build the Clinical Co-Pilot |

## Stage 1 — Run It Locally

Get OpenEMR running in your local development environment with realistic sample patient data. You cannot audit or build what you cannot run. Document your setup process — this becomes part of your README and your understanding of the system's dependencies.

## Stage 2 — Deploy It

Deploy your fork to a publicly accessible environment. It does not need to be production-hardened at this stage, but it must be live and reachable. You will deploy the final agent to the same infrastructure, so choose your stack thoughtfully.

**Hard Gate:** You must submit your deployed app's url as part of every submission.

## Stage 3 — Audit It

Before considering any additions to the project, you must complete a full audit of the system. This audit should contain at least the following parts:

- **Security audit** — identify authentication and authorization risks, data exposure vectors, PHI handling issues, and any HIPAA-relevant gaps in the current system.
- **Performance audit** — understand where the system is slow, what the bottlenecks are, how the data is structured, and what constraints will affect your agent's response latency.
- **Architecture audit** — document how the system is organized, where data lives, how the layers interact, and what the integration points are for adding new capabilities.
- **Data Quality audit** — find how complete, consistent, and reliable that data actually is. Missing fields, inconsistent formatting, duplicate records, and stale data all become agent failure modes.
- **Compliance & Regulatory audit** — HIPAA is mentioned in the security audit, but it deserves its own pass: audit logging requirements, data retention policies, breach notification obligations, and BAA (Business Associate Agreement) implications of sending PHI to an LLM provider.

**Hard Gate:** a markdown document (./AUDIT.md) with all of your audit findings. The document *must begin* with a one page summary (~500 words) of your key findings. The summary's brevity requirement is intentional. The summary should highlight the most impactful audit findings, not just a dump of everything you found.

## Stage 4 — Create User Profiles and Use Cases

Before you can plan an agent, decide who it's actually for and what specific problem it solves for them. "Physicians need help finding information" is not a user definition. It is a thesis statement that has produced a thousand failed health-tech products.

Pick a real, narrow user: a primary care physician with a 20-patient day, an ED resident on overnight intake, a hospitalist rounding on twelve admissions before noon. These are different people with different workflows, different pain points, and different tolerances for an agent's behavior. The user you choose constrains everything that follows, such as what data the agent needs, how fast it has to respond, what it should refuse to do, and what "useful" even means.

Then ground that user in a concrete workflow. Walk through the moment your agent enters their day: what are they doing in the thirty seconds before they open it, what do they need from it, and what do they do with the output? Identify specific use cases; not "answer questions about a patient," but "between 8:50 and 9:00 AM, surface what's changed for each patient on today's schedule and flag anything that needs attention." And for each, be ready to defend why a conversational agent is the right shape — not a dashboard, not a sorted list, not a better chart view.

The bar is not that an agent is technically possible. The bar is that the agent is the thing the user would actually choose.

**Hard Gate:** a markdown document (./USERS.md) defining your target user, their workflow, and specific use cases your agent addresses. Each use case must include an explicit answer to *why an agent is the right solution here*. This document is the source of truth your ARCHITECTURE.md must trace back to. Every agent capability you build in Stage 5 must point to a use case here.

## Stage 5 — Develop the AI Integration Plan

Using the findings from your AUDIT.md as input, you must synthesize your findings into a forward-looking roadmap: where will your agent live, how will it access patient data, what are the authorization boundaries, what are the risks, and how will you address them?

You do not need to implement anything at this stage. You need to think clearly, write it down, and be able to defend it (you will on Tuesday). This plan becomes your roadmap for Early Submission.

**Hard Gate:** a markdown document (./ARCHITECTURE.md) outlining how you intend to build your agent to address the case study. The document *must also begin* with a one page summary (~500 words) of your high level architecture. The summary should highlight key decisions, major considerations, and tradeoffs.

# Agent Requirements

The following are the required components of your agent. How you implement each one is a design decision you own.

## Agentic Chatbot

The core interface is a conversational agent. It is not a search bar, a dashboard widget, or a report generator — it is a multi-turn AI agent that can receive follow-up questions, maintain context across a conversation, and invoke tools to retrieve and reason over patient data.

Every agent capability you build must trace to a specific user problem you identified. If you cannot point to a use case in your USERS.md that requires multi-turn conversation, you should not have multi-turn conversation. If you cannot point to a use case that requires tool chaining, you should not have tool chaining. The agent's surface area is determined by the user's needs, not by what's technically interesting to build.

## Verification System

Every response the agent produces must pass through a verification layer before it reaches the user. The purpose of this layer is to ensure that what the agent says is actually supported by the patient's data.

Verification in this context means two things:

- Source attribution — the agent's claims must be traceable to specific records in the patient's file. If a claim cannot be attributed to a source, it should not be stated as fact.
- Domain constraint enforcement — the agent must be aware of clinical rules and flag or reject responses that violate them. What those constraints are, and how you enforce them, is your design problem to solve.

Think carefully about where in the agent's flow verification happens, what it catches, and what it doesn't. Document your approach and its known limitations.

## Observability

You cannot improve what you cannot see. Implement observability from the start — not as an afterthought. At minimum, you should be able to answer these questions from your logs at any time:

- What did the agent do on a specific request, and in what order?
- How long did each step take?
- Did any tools fail, and if so, why?
- How many tokens were consumed, and at what cost?

What tool you use, what metrics you track beyond the minimum, and how you visualize or surface that data is up to you. The requirement is that observability is real, wired in from the beginning, and used — not just installed.

### Evaluation

Build a test suite that lets you measure whether your agent is working. What you test, how many cases you build, and how you define pass/fail are decisions you make — but those decisions must be intentional and defensible.

A strong eval suite does more than confirm happy paths. It surfaces failure modes, regression risks, and the edge cases that matter in clinical settings: missing data, ambiguous queries, inputs that attempt to extract information the requester is not authorized to see.

---

## Submission Requirements

Final deadline: Sunday 10:59 PM CT

| Deliverable | Requirements |
|---|---|
| **GitHub Repository** | Forked from OpenEMR. Includes setup guide, architecture overview, and deployed link. |
| **Audit Document (./AUDIT.md)** | All audit findings with a 1-page (~500 word) summary detailing key findings. |
| **User Doc (./USER.md)** | The user you're focusing on with a list of use cases that your agent will address. |
| **Agent Architecture Doc (./ARCHITECTURE.md)** | Your plan to integrate AI with technical details such as (but not limited to) framework choices, verification strategy, and known tradeoffs. Must begin with a 1-page (~500 word) summary of a high-level overview with key decisions. |
| **Demo Video (3–5 min)** | One demo video with each submission showcasing the work you've done, highlighting key decisions and showcasing the product. |
| **Eval Dataset** | Your test suite with results. Structure and scope are your design decisions. |
| **AI Cost Analysis** | Actual dev spend and projected production costs at 100 / 1K / 10K / 100K users. Also consider architectural changes needed at each level. This is not simply cost-per-token * n users. |
| **Deployed Application** | Publicly accessible. For early and final submissions, the agent must work in the live environment. |
| **Social Post (Final submission only)** | Share on X or LinkedIn: describe the project, show the agent, tag @GauntletAI. |

# Interview Preparation

Austin admission requires an interview with each major deliverable. You will be expected to discuss your work at depth. Prepare to speak to the following.

## Your Audit

- Walk us through your most important finding.
- What would you have missed if you had skipped the audit and gone straight to building?
- How did the audit change your AI integration plan?

## Your Architecture

- Why did you design the verification layer the way you did?
- What does your agent do when a tool fails or a record is missing?
- Where are the trust boundaries in your system, and how are they enforced?

## Your Evaluation

- What does your eval suite test that a happy-path demo would not reveal?
- What did you find when you ran it?
- What would you add to it next?

## Production Thinking

- How would you scale this to a 500-bed hospital with 300 concurrent clinical users?
- What would you need to change before you'd be comfortable with a real physician relying on this?
- What failure mode worries you most, and why?

# Final Note

*The deliverable that matters is not the one that looks most impressive in a demo. It's the one you could defend in front of a hospital CTO who is deciding whether to put it in front of their physicians.*

That is the standard. Build to it.

# Appendix: Pre-Search Checklist

Use this list to ensure you've thought through a variety of perspectives in your planning.

## Phase 1: Define Your Constraints

### 1. Domain Selection

- What specific use cases will you support?
- What are the verification requirements for this domain?
- What data sources will you need access to?

### 2. Scale & Performance

- Expected query volume?
- Acceptable latency for responses?
- Concurrent user requirements?
- Cost constraints for LLM calls?

### 3. Reliability Requirements

- What's the cost of a wrong answer in your domain?
- What verification is non-negotiable?
- Human-in-the-loop requirements?
- Audit/compliance needs?

### 4. Team & Skill Constraints

- Familiarity with agent frameworks?
- Experience with your chosen domain?
- Comfort with eval/testing frameworks?

## Phase 2: Architecture Discovery

### 5. Agent Framework Selection

- Single agent or multi-agent architecture?
- State management requirements?
- Tool integration complexity?

### 6. LLM Selection

- OpenAI vs Claude vs open source?
- Structured output support requirements?
- Context window needs?
- Cost per query acceptable?

### 7. Tool Design

- What tools does your agent need?
- External API dependencies?
- Mock vs real data for development?
- Error handling per tool?

### 8. Observability Strategy

- LangSmith vs Langfuse vs Braintrust vs other?
- What metrics matter most?
- Real-time monitoring needs?
- Cost tracking requirements?

### 9. Eval Approach

- How will you measure correctness?
- Ground truth data sources?
- Automated vs human evaluation?
- CI integration for eval runs?

### 10. Verification Design

- What claims must be verified?
- Fact-checking data sources?
- Confidence thresholds?
- Escalation triggers?

## Phase 3: Post-Stack Refinement

### 11. Failure Mode Analysis

- What happens when tools fail?
- How to handle ambiguous queries?
- Rate limiting and fallback strategies?
- Graceful degradation approach?

### 12. Security Considerations

- Prompt injection prevention?
- Data leakage risks?
- API key management?
- Audit logging requirements?

### 13. Testing Strategy

- Unit tests for tools?
- Integration tests for agent flows?
- Adversarial testing approach?
- Regression testing setup?

## 14. Open Source Planning

- What will you release?
- Licensing considerations?
- Documentation requirements?
- Community engagement plan?

## 15. Deployment & Operations

- Hosting approach?
- CI/CD for agent updates?
- Monitoring and alerting?
- Rollback strategy?

## 16. Iteration Planning

- How will you collect user feedback?
- Eval-driven improvement cycle?
- Feature prioritization approach?
- Long-term maintenance plan?